

AI Smart Tutor

using Advanced Prompt Engineering
Techniques

**Rancang Bangun Website AI Smart Tutor sebagai Media
Tanya Jawab Pembelajaran Interaktif**

AI Smart Tutor Website as an Interactive Learning Question-Answering Media

Prepared by: Ivansyah Putra

Institution: STMIK Kaputama Binjai

Class/Course: Google Prompt Engineering

Final Project Presentation



Project Overview

The website combines AI-powered tutoring, structured output, and adaptive guidance.

Project Title

AI Smart Tutor Website as an Interactive Learning Question-Answering Media

A web-based educational chatbot that helps users ask academic questions, generate study plans, create quizzes, and evaluate answers using Gemini API.

4

AI Tutor Modes

3

Learning Levels

Main output:

Interactive chatbot website with prompt-optimized educational responses.



Problem Statement & Objectives

The project addresses learning support needs with AI-powered, structured guidance.

Problems

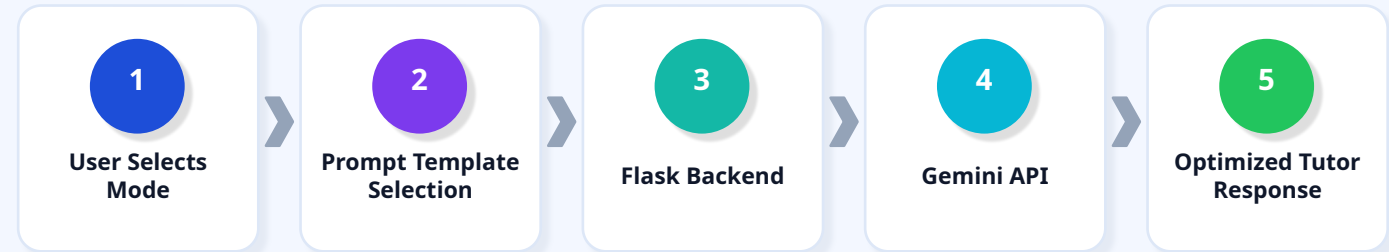
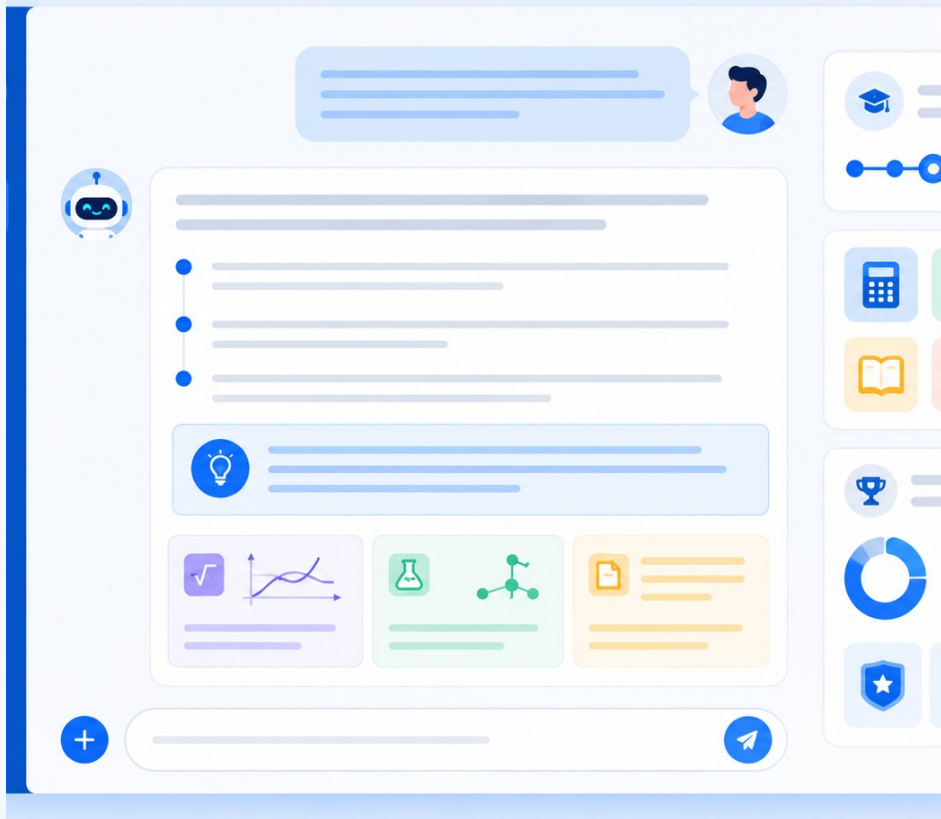
- 1 Students need quick explanations for academic topics.
- 2 AI responses can be inconsistent without prompt control.
- 3 Different learners require different explanation depth.
- 4 Self-study needs quizzes, plans, and answer feedback.

Objectives

- 1 Build a Python Flask chatbot website using Gemini API.
- 2 Generate structured and context-aware educational answers.
- 3 Support Beginner, Intermediate, and Advanced levels.
- 4 Add study plan, quiz, evaluator, and prompt optimization features.

System Architecture & Workflow

A lightweight Flask backend connects the web interface to Gemini API.



Data Approach

The project does not use a database. Chat history is stored locally in the browser using Local Storage, while API credentials remain in the Flask backend environment file.

AI Tutor Modes

Each mode uses a task-specific prompt template to control output quality.

01

Question Answering

Structured academic answers with context-aware explanation.

02

Study Plan Generator

Day-by-day learning plan adjusted to the selected level.

03

Quiz Generator

Practice questions and answer keys for self-assessment.

04

Answer Evaluator

Score, strengths, weaknesses, corrected answer, and tips.



Why multiple modes?

A single generic prompt often produces inconsistent responses. Mode-based prompting makes the AI focus on a specific learning task and produce predictable output structures.

Result: clearer answers, better learning support, and more reliable demo output.

Advanced Prompt Engineering Strategy

The system uses prompt structure to control AI behavior and response quality.

1

Role-Based Prompting

AI acts as tutor, study planner, quiz maker, or examiner.

2

Structured Output Formatting

Each mode follows a consistent numbered response format.

3

Learning-Level Adaptation

Beginner, Intermediate, and Advanced responses are adjusted.

4

Task-Specific Templates

Each AI Tutor Mode has a dedicated prompt.

5

Prompt Optimization

Final prompts improve clarity, relevance, and consistency.

6

Safety Boundary

The chatbot is guided to answer educational topics only.

Prompt Comparison Engine & Optimization

The website explains how prompt templates are compared by task focus and expected output.

AI Tutor Mode	Prompt Focus	Expected Output
Question Answering	Structured academic explanation	Answer, context explanation, concept explanation, guidance
Study Plan Generator	Learning schedule creation	Day-by-day plan with personalized guidance
Quiz Generator	Practice question generation	Quiz questions, answer key, and learning tip
Answer Evaluator	Examiner-style assessment	Score, strengths, weaknesses, corrected answer

Optimization method: role instruction + task-specific structure + learning level rules + fixed output sections.

Implementation Stack

The system is built with a simple, lightweight web architecture.

Frontend

HTML, CSS, JavaScript

Backend

Python Flask

AI Model

Gemini API

Security

.env for API Key

Storage

Browser Local Storage

Output

Interactive chatbot website

Key principle: No database required; project remains simple, lightweight, and stable for demo.



Testing & Result Summary

Testing confirmed that all core modes work as expected.

Functional Test Cases

● Question Answering	What is database?	Structured answer generated
● Study Plan Generator	5-day HTML & CSS plan	Study plan generated
● Quiz Generator	5 questions about information systems	Questions + answer key generated
● Answer Evaluator	Question + user answer	Score + feedback generated
● Chat History	Refresh browser	Messages remain stored locally

Result: Website is ready for demo and deliverable submission.



WBS Alignment

The website covers all required milestones from project setup to final presentation.



Project setup and prompt exploration

Flask project, Gemini API, initial prompt design



Structured answer and multi-level explanation

Response format + Beginner/Intermediate/Advanced



Study plan and quiz generator

Dedicated modes for plans and quizzes



Answer evaluator

Score, strengths, weaknesses, corrected answer



Prompt comparison and optimization

Mode-based prompt templates and optimization table



Finalization and presentation

Final website, docs, slides, demo plan

Recommended Demo Flow

A simple recording flow that shows every required feature.

- 1 Open Website** Show landing page, feature cards, and Project Information section.
- 2 Question Answering** Ask: "What is database?" using Beginner level.
- 3 Study Plan Generator** Ask for a 5-day HTML and CSS study plan.
- 4 Quiz Generator** Create 5 quiz questions about information systems.
- 5 Answer Evaluator** Submit a sample answer and show score + feedback.
- 6 History + Explanation** Refresh page to show history, then open Project Explanation.

Final Result

AI Smart Tutor is ready for submission.

The project delivers a complete educational chatbot website aligned with WBS M1–M6, using Python Flask, Gemini API, mode-based prompt engineering, and local chat history.

Key outcomes

- Interactive chatbot interface
- Four AI Tutor modes
- Learning-level personalization
- Prompt comparison and optimization
- Demo-ready project documentation